

Using AJAX

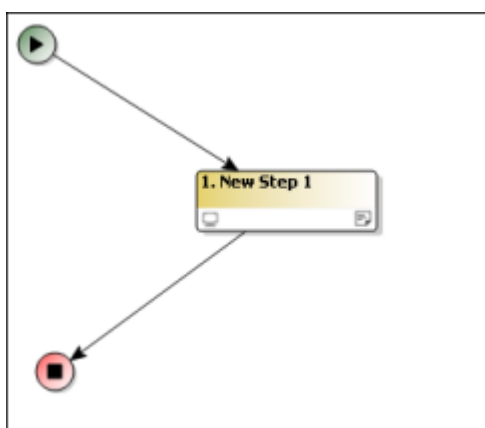
It is possible to call a DFC from a screen asynchronously (an AJAX call from the scripts). Follow the steps in the example below to use this functionality.

Security Constraints

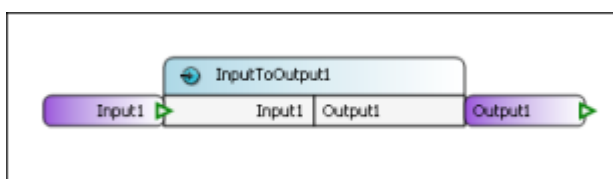
Only DFCs that appear in the **Invoke DFCs/AJAX** section on the **Advanced** tab in the **Properties** pane can be invoked from a specific DFC step.

Example

1. Create a DFC that will be called asynchronously (AJAX DFC).
 - a. Link **New Step 1** to the end node and enter the step name to edit it.



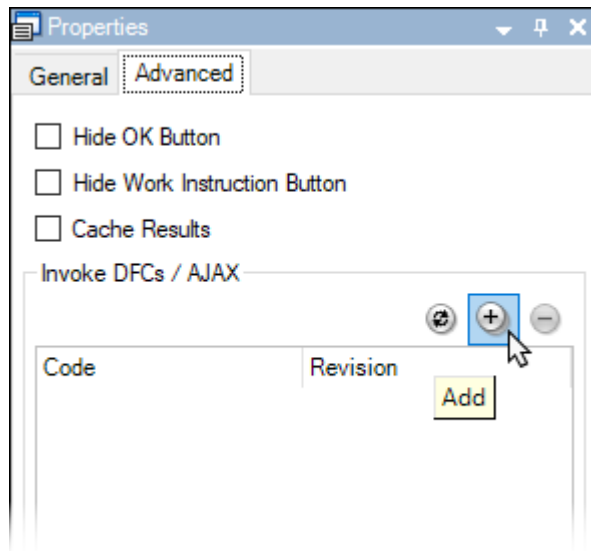
- b. Add one function of type **Input to Output** with an external input and output.



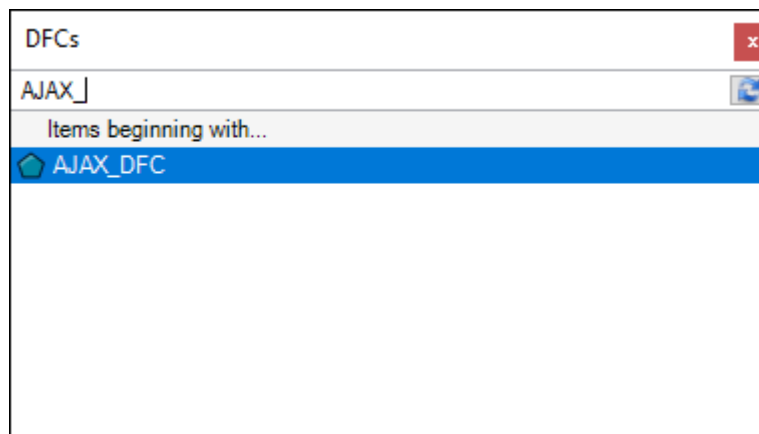
- c. Change the DFC status to prototype.

Only DFCs in active or prototype status can be called asynchronously.

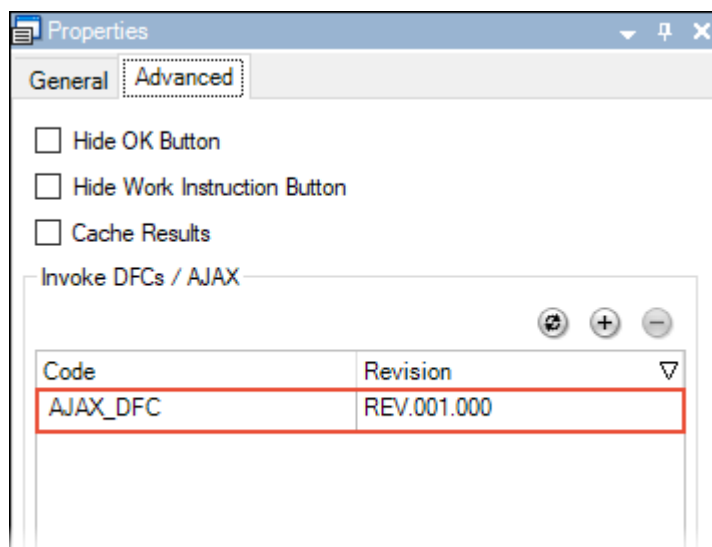
2. Create the second DFC that will call your AJAX DFC.
 - a. Link **New Step 1** to the End Node, enter the name of the step to edit it. Switch to the HTML Layout Editor (use the right-click menu and choose **Convert to HTML Layout**).
 - b. Navigate to the Advanced tab on the Properties pane and add the previously created AJAX DFC – click **+** (Add) in the **Invoke DFCs/AJAX** section and choose the AJAX DFC from the **DFCs** lookup window).



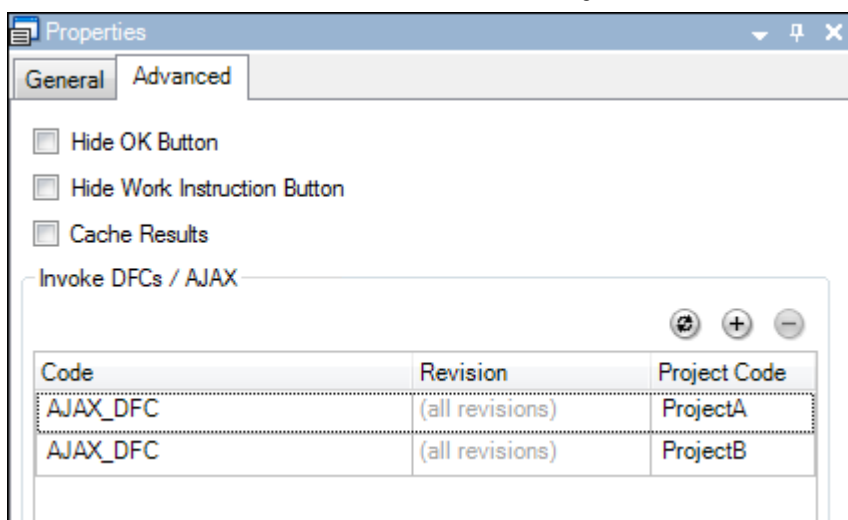
AJAX DFCs are only accessible when the HTML Layout Editor is used.



The DFC will appear in the **Invoke DFCs/AJAX** section.

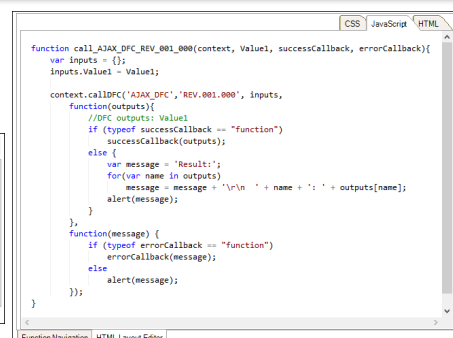
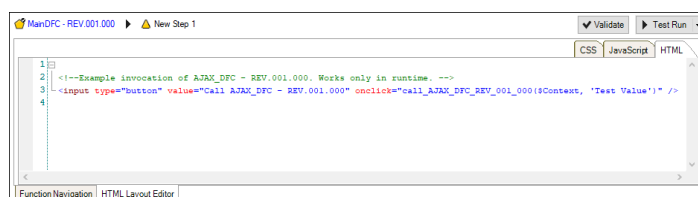


The additional **Project Code** column appears only for **Project DFCs** in the Invoke DFCs / AJAX section.

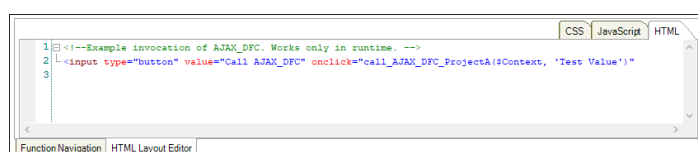


c. Drag the AJAX DFC to the HTML Layout Editor (HTML tab).

A button with this DFC is created in the HTML Layout Editor and a JavaScript code is generated. The generated code is fully functional, and both parts from JavaScript and HTML may be modified according to specific needs. The refresh button does not update the list of External Inputs and Outputs in the generated code.

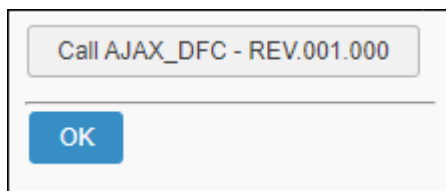


In the case of **Project DFCs**, the code is as follows:

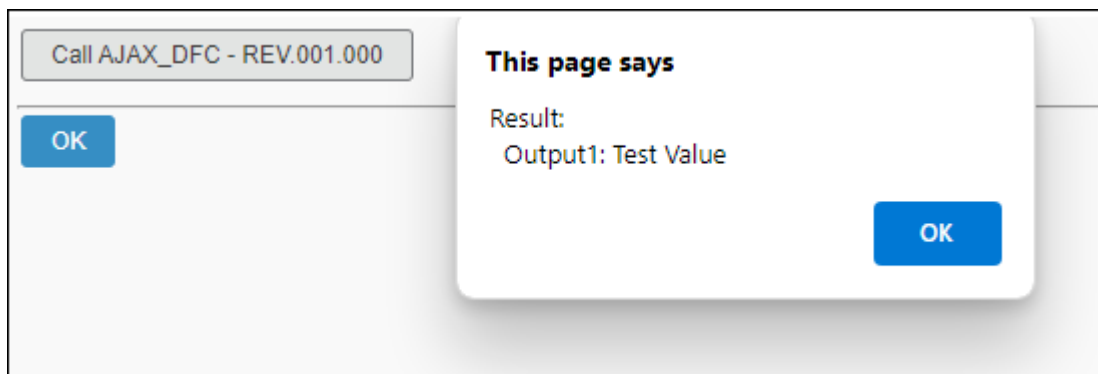


You may use any JavaScript object as an Input. When the DFC contains the Input of the Complex data type, it will be deserialized.

d. Start the Test Run.



- e. Click the **Call AJAX DFC** button. It will display a pop-up window with Outputs by default.



- f. You can modify the processing of the Outputs according to your needs. There are the following options:
- i. Passing Functions (on successCallback, on errorCallback) in the Function call (3rd and 4th parameter, marked in blue).
 - ii. Implementing on success, on error handling directly in the JavaScript Function definition (JavaScript tab, marked in green).

```
function call_AJAX_DFC_REV_001_000(context, Input1,
successCallback,errorCallback){
    var inputs = {};
    inputs.Input1 = Input1;
    context.callDFC('AJAX_DFC','REV.001.000', inputs,
        function(outputs){
//DFC outputs: Output1
            if (typeof successCallback == "function")
                successCallback(outputs);
            else {
                var message = 'Result: ';
                for(var name in outputs)
                    message = message + '\r\n  ' + name + ': ' +
outputs[name];
                alert(message);
            }
        },
        function(message) {
            if (typeof errorCallback == "function")
                errorCallback(outputs);
            else
                alert(message);
        });
}
```

- iii. Using async/await syntax:

```
async functioncall_AJAX_DFC_REV_001_000(context, input1){
  var inputs = {};

  inputs.input1 = input1;

  try {
    var result = await context.callDFC
    ('AJAX_DFC','REV.001.000', inputs);
    var message = 'Result: ';

    for(var name in result) {
      message = message + '\r\n ' + name + ': ' +
result[name];
    }
    alert(message);
  } catch (ex) {
    console.log(ex);
  }
}
```

When the response message comes from the literal, its ID is accessible from `message.literalId` for `errorCallback` and `ex.literalId` for `catch`.